Assistant de recherche d'images

LLM (Ollama) + serveur MCP + ResNet-18 fine-tuné
sur Oxford-IIIT Pets

Louai Boudrahem

Zakaria Errachidi

Abderrahmane Guemmar

Novembre 2026

Dépôt : `github.com/LouaiBoudrahem/dp-image-search`

Table des matières

1 Introduction **2**

2 Installation et mode d'emploi **3**

2.1 Installation 3

2.2 Mode d'emploi 4

3 Méthodologie **5**

3.1 LLM et prompt système 5

3.2 Modèle spécialiste : entraînement 5

3.3 Modèle spécialiste : évaluation 6

3.4 Serveur MCP et les 5 outils 10

4 Développement : difficultés rencontrées et solutions **12**

5 Conclusion **13**

5.1 Compétences acquises 13

5.2 Pistes d'amélioration 13

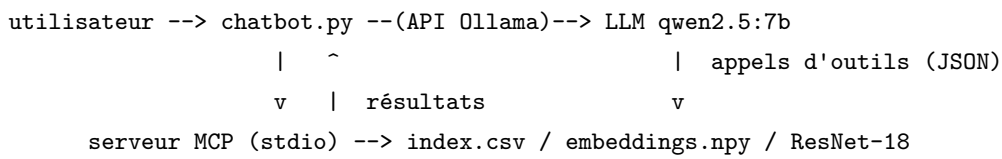
1. Introduction

L'objectif du projet est de construire un assistant conversationnel capable d'explorer une collection d'images en langage naturel. L'utilisateur pose une question en français (« Montre-moi des chats persans prédits avec plus de 90 % de confiance », « Avec quelle race le modèle confond-il le Ragdoll ? »). Un grand modèle de langage (LLM) servi localement par Ollama interprète la demande et appelle des outils exposés par un serveur MCP (Model Context Protocol). Ces outils s'appuient sur un modèle spécialiste de vision, un ResNet-18 fine-tuné pour reconnaître les races d'animaux.

Le jeu de données est Oxford-IIIT Pets : 7 349 photos de 37 races (12 de chats, 25 de chiens), découpées en 3 680 images trainval et 3 669 images de test. Le modèle doit distinguer des races parfois très proches (American Pit Bull Terrier et Staffordshire Bull Terrier, Birman et Ragdoll), ce qui en fait un problème de classification à grain fin.

L'architecture comporte trois briques :

- **Modèle spécialiste** (`train.py`, `evaluate.py`) : un ResNet-18 entraîné puis exporté sous forme d'un index des prédictions (`index.csv`), d'embeddings de dimension 512 (`embeddings.npy`) et de la liste des classes (`classes.json`).
- **Serveur MCP** (`mcp_server.py`) : il charge ces fichiers et expose 5 outils (`classify`, `find_similar`, `filter_images`, `count_by`, `most_confused`).
- **Chatbot** (`chatbot.py`) : un client qui lance le serveur MCP en sous-processus, transmet la liste des outils au LLM et exécute la boucle d'appels d'outils.



Le rapport présente l'installation (partie 2), puis la méthodologie : LLM et prompt, entraînement et évaluation du modèle, serveur MCP (partie 3). Viennent ensuite les difficultés rencontrées (partie 4) et un bilan (partie 5).

2. Installation et mode d'emploi

2.1 Installation

Le projet nécessite Python 3 et Ollama. On crée un environnement virtuel puis on installe les dépendances listées dans le README :

```
python3 -m venv .venv && source .venv/bin/activate
# Windows : .venv\Scripts\activate
pip install torch torchvision scikit-learn matplotlib numpy \
            pillow ollama "mcp<2"
ollama pull qwen2.5:7b      # LLM par défaut
```

Sur un GPU NVIDIA, on installe la version CUDA de PyTorch. Le code choisit automatiquement le périphérique : CUDA, puis MPS (puces Apple), puis CPU. Les 6 checkpoints entraînés (`models/*.pt`, 43 Mo chacun) et les fichiers exportés dans `outputs/` sont inclus dans le dépôt : le chatbot fonctionne donc sans réentraînement.

2.2 Mode d'emploi

Commande	**Rôle**
`python train.py [--seed N] [--scratch]`	Entraîne un ResNet-18 (pré-entraîné par défaut) ; ajoute une ligne à `results.csv` et sauvegarde `models/<run>.pt`
`run_all.bat`	Lance les 6 entraînements (3 seeds × pré-entraîné / from scratch)
`python evaluate.py`	Évalue tous les checkpoints, calcule les baselines et produit `summary.txt`, le rapport par classe, les figures et les fichiers du serveur MCP
`python mcp_server.py --test`	Auto-test : appelle les 5 outils sans LLM
`python chatbot.py`	Chatbot interactif (modèle local qwen2.5 :7b)
`python chatbot.py --model llama3.1:8b --host http://HOTE:11434`	Utilise un serveur Ollama distant (par exemple celui du CRIL)
`python eval_chatbot.py`	Évalue le taux de bons appels d'outils sur 20 requêtes de test

Tab. 1. Commandes disponibles.

`train.py` et `evaluate.py` téléchargent automatiquement le jeu de données dans `data/` (via torchvision). Exemple de session avec le chatbot :

```
vous > Avec quelles races le modèle confond-il le plus
       le Staffordshire Bull Terrier ?
  [tool] most_confused({"label": "Staffordshire Bull Terrier"})
bot  > Le modèle le confond surtout avec l'American Bulldog et
       l'American Pit Bull Terrier. Hypothèse : morphologie
       musclée, tête large et robe courte similaires.
```

3. Méthodologie

3.1 LLM et prompt système

Le LLM par défaut est qwen2.5 :7b, exécuté localement par Ollama. Nous l'avons choisi parce qu'il gère nativement l'appel de fonctions (*tool calling*) et qu'il tourne sur une machine personnelle. llama3.1 :8b peut aussi être utilisé via le serveur Ollama du CRIL (option `--host` ou variable `OLLAMA_HOST`). La température est fixée à 0 pour obtenir des réponses reproductibles, ce qui compte pour l'évaluation.

Le client récupère la liste des outils auprès du serveur MCP (`list_tools`) et convertit leurs schémas JSON au format attendu par Ollama. La fonction `ask` implémente une boucle : tant que le LLM demande des outils, chaque appel est exécuté via `session.call_tool` et son résultat est ajouté à la conversation avec le rôle `tool`. La boucle s'arrête quand le LLM produit une réponse textuelle, ou au bout de 6 étapes. Ce mécanisme permet d'enchaîner plusieurs outils.

Le prompt système (en anglais, car les modèles suivent mieux les consignes d'outils dans cette langue) impose les règles suivantes :

- utiliser obligatoirement les outils et ne jamais inventer de chiffres, de races ni de noms d'images ;
- décrire brièvement les 5 outils et le format d'une image (id, nom de fichier ou chemin) ;
- convertir les pourcentages en nombres entre 0 et 1 (80 % $\rightarrow$ 0.8) ;
- enchaîner les outils en réutilisant la sortie de l'un comme entrée du suivant ;
- en cas d'erreur d'un outil, le signaler et proposer une correction (par exemple les noms de races valides) ;
- répondre dans la langue de l'utilisateur et, pour un « pourquoi », donner une hypothèse courte (pelage, couleur, morphologie) en précisant qu'il s'agit d'une hypothèse.

3.2 Modèle spécialiste : entraînement

Le modèle est un ResNet-18 (torchvision) dont la dernière couche est remplacée par une couche linéaire 512 $\rightarrow$ 37. Nous comparons deux variantes (ablation) :

	Pré-entraîné (ImageNet)	**From scratch**
Initialisation	poids `IMAGENET1K_V1`	aléatoire
Époques	10	30
Optimiseur	Adam, lr 10^{-4} (backbone) et 10^{-3} (tête, $\times 10$)	Adam, lr 10^{-3}
Planification	cosine annealing	cosine annealing
Batch / augmentation	32 / RandomResizedCrop(224) + flip horizontal	idem

Tab. 2. Paramètres d'entraînement des deux variantes.

L'ensemble trainval est découpé en 85 % d'entraînement et 15 % de validation selon une permutation qui dépend de la seed. À chaque époque, on garde le checkpoint ayant la meilleure précision de validation ; l'ensemble de test n'est utilisé qu'à la fin. Les images de validation et de test sont redimensionnées à 256 puis recadrées au centre (224) et normalisées avec les statistiques d'ImageNet. Chaque variante est entraînée avec 3 seeds (0, 1, 2), soit 6 runs.

Le taux d'apprentissage plus faible pour le backbone pré-entraîné évite de détruire les caractéristiques apprises sur ImageNet, tandis que la nouvelle tête apprend plus vite.

3.3 Modèle spécialiste : évaluation

`evaluate.py` évalue chaque checkpoint sur les 3 669 images de test avec trois métriques : la précision top-1 et top-5 en classification, et la precision@5 en recherche d'images. Cette dernière est la part des 5 plus proches voisins (similarité cosinus des embeddings de 512 dimensions, image requête exclue) qui sont de la même race : elle mesure directement la qualité de l'outil `find_similar`. Deux baselines servent de point de comparaison : la classe majoritaire et une recherche sur les pixels bruts (images 32×32 aplaties).

Modèle (test, 3 669 images)	**Top-1**	**Top-5**	**Precision@5**
ResNet-18 pré-entraîné (3 seeds)	90,0 ± 0,2 %	99,2 ± 0,1 %	89,3 ± 0,3 %
ResNet-18 from scratch (3 seeds)	40,9 ± 0,7 %	76,8 ± 0,1 %	30,9 ± 0,9 %
Baseline classe majoritaire (Abyssinian)	2,7 %	–	–
Baseline pixels bruts 32×32	–	–	5,9 %

Tab. 3. Résultats sur l'ensemble de test (moyenne ± écart-type sur 3 seeds, `outputs/summary.txt`).

Run	Époques	Val top-1	Test top-1	Test top-5
pretrained_seed0	10	94,2 %	89,9 %	99,3 %
pretrained_seed1	10	92,9 %	89,8 %	99,2 %
pretrained_seed2	10	93,7 %	90,2 %	99,1 %
scratch_seed0	30	49,5 %	41,3 %	76,9 %
scratch_seed1	30	46,9 %	40,1 %	76,7 %
scratch_seed2	30	51,3 %	41,3 %	76,8 %

Tab. 4. Détail par seed (`results.csv`). La validation est légèrement optimiste par rapport au test, puisqu'elle sert aussi à choisir le checkpoint.

Ablation pré-entraîné vs from scratch. Le pré-entraînement sur ImageNet apporte +49,1 points de top-1 (90,0 % contre 40,9 %) avec trois fois moins d'époques. Avec environ 3 100 images d'entraînement pour 37 classes (une centaine par race), un réseau de 11 millions de paramètres ne peut pas apprendre seul des caractéristiques visuelles assez fines.

L'écart est encore plus marqué pour la recherche d'images (precision@5 de 89,3 % contre 30,9 %) : les embeddings du modèle pré-entraîné regroupent bien les images d'une même race, ce qui justifie leur usage dans `find_similar`. Les écarts-types sont faibles ($\leq$ 0,9 point) : les résultats sont stables d'une seed à l'autre et la différence entre les variantes est largement significative. Les deux modèles dépassent nettement les baselines (2,7 % et 5,9 %). Le modèle from scratch apprend donc bien quelque chose, mais il reste loin du transfert.

Analyse par classe (modèle pré-entraîné, seed 0, utilisé par le serveur MCP). Les races les mieux reconnues ont un aspect très distinctif : Keeshond (F1 = 0,995), Leonberger (0,985), Pug (0,980). Les plus difficiles sont les chiens de type bull-terrier, American Pit Bull Terrier (F1 = 0,622, rappel 0,51) et Staffordshire Bull Terrier (0,638), ainsi que le Ragdoll (0,766).

Les confusions les plus fréquentes sont American Pit Bull Terrier → Staffordshire (15), Egyptian Mau → Bengal (14), Birman → Ragdoll (14), American Pit Bull Terrier → American Bulldog (13) et Staffordshire → American Bulldog (12). Ce sont des races réellement proches, et les confusions vont souvent dans les deux sens (Ragdoll → Birman : 9).

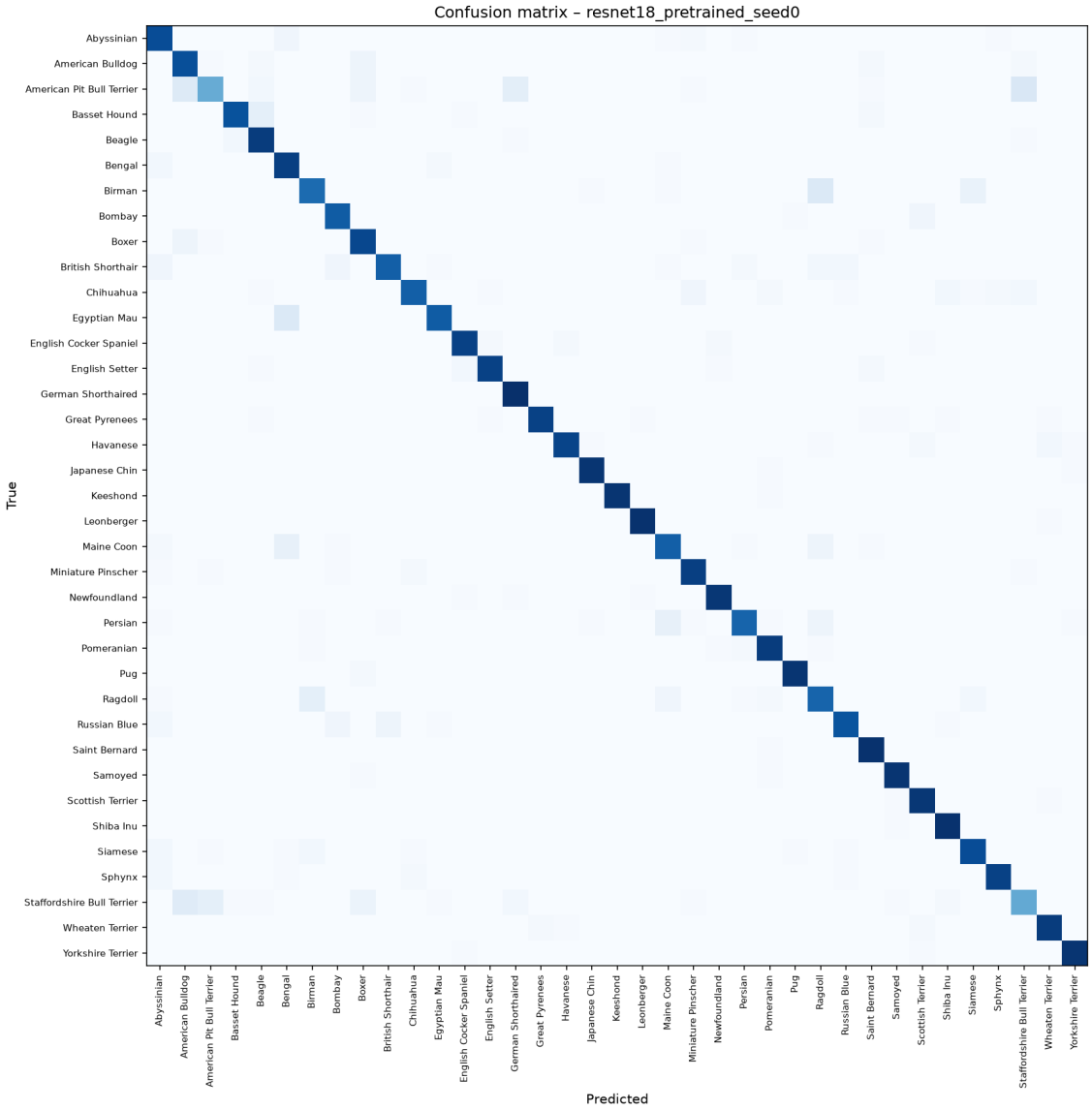


Fig. 1. Matrice de confusion (ResNet-18 pré-entraîné, seed 0). La diagonale est très marquée ; les principaux blocs hors diagonale regroupent les bull-terriers (en bas à gauche) et les paires Birman/Ragdoll et Egyptian Mau/Bengal.

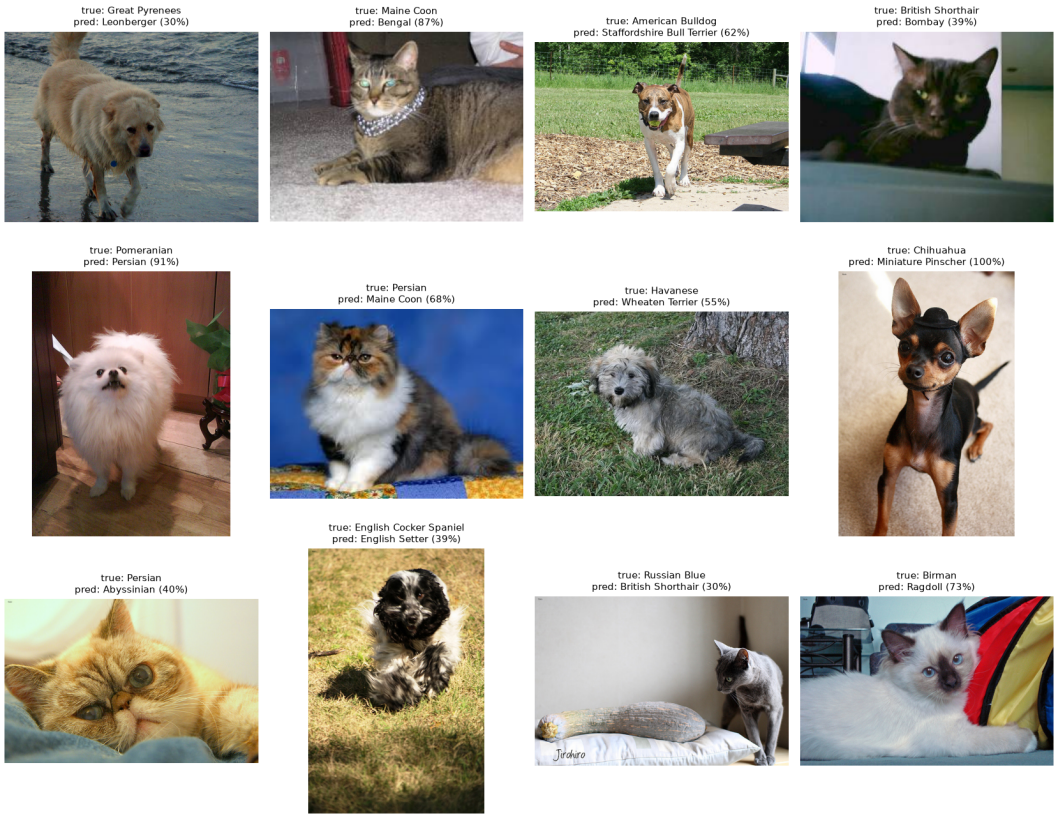


Fig. 2. 12 erreurs tirées au hasard dans l'ensemble de test (vraie race, race prédite et confiance).

Exemples d'erreurs commentés (figure 2) :

- **Chihuahua → Miniature Pinscher (100 %)** : chien à poil court noir et feu, oreilles dressées. Ce patron de couleur est typique du Pinscher nain ; l'erreur est compréhensible, mais la confiance de 100 % montre que le modèle peut être très sûr de lui à tort.
- **Birman → Ragdoll (73 %)** : deux chats au pelage colourpoint et aux yeux bleus. Le critère distinctif du Birman, les « gants » blancs aux pattes, est peu visible sur la photo.
- **Pomeranian → Persian (91 %)** : boule de poils blanche, photographiée de face. L'erreur traverse la frontière chien/chat : le modèle s'appuie surtout sur la texture du pelage plutôt que sur la forme.
- **Persian → Maine Coon (68 %)** et **Maine Coon → Bengal (87 %)** : poil long tricolore dans le premier cas, robe tigrée dans le second. Le motif de la robe domine la décision.
- **Great Pyrenees → Leonberger (30 %)** et **Russian Blue → British Shorthair (30 %)** : erreurs à faible confiance. Pelage mouillé à la plage pour l'un, chat petit dans

l'image pour l'autre. Dans ces cas, le top-5 (99,2 %) contient presque toujours la bonne race.

3.4 Serveur MCP et les 5 outils

Le serveur est écrit avec FastMCP (SDK Python `mcp`) et communique avec le chatbot par *stdio*. Au démarrage, il charge `index.csv` (chemin, split, vraie race, race prédite et confiance pour les 7 349 images), `classes.json` et `embeddings.npy`, dont les lignes sont normalisées une fois pour toutes : la similarité cosinus se réduit alors à un produit matriciel. Le ResNet-18 n'est chargé qu'à la demande (*lazy loading*), quand il faut traiter une nouvelle image.

Deux fonctions utilitaires rendent les outils tolérants aux formulations du LLM :

- `find_class` retrouve une race à partir d'un nom approximatif : correspondance exacte (sans tenir compte de la casse ni des « _ »), puis sous-chaîne unique (« staffordshire »), puis recherche floue `difflib` (seuil 0,6) ;
- `resolve_image` accepte un identifiant numérique, un nom de fichier (« Abyssinian_100 ») ou le chemin d'une nouvelle image, qui est alors passée dans le modèle.

Outil	**Entrées**	**Sortie**
`classify`	image	Top-3 des races avec leur probabilité (passage dans le ResNet-18)
`find_similar`	image, k (défaut 5, max 50)	Les k images les plus proches (similarité cosinus des embeddings)
`filter_images`	label, min_confidence, max_results	Nombre total et exemples d'images prédites comme cette race au-dessus du seuil (accepte 0.9 ou 90)
`count_by`	label (optionnel)	Nombre d'images prédites pour une race, ou pour les 37 races
`most_confused`	label	Sur le test : races avec lesquelles la race est confondue, dans les deux sens, avec des exemples d'erreurs

Tab. 5. Les 5 outils exposés par le serveur MCP.

Les erreurs (race inconnue, image introuvable) sont renvoyées sous la forme d'un objet `{"error": ...}` qui contient par exemple la liste des races valides, pour que le LLM puisse se corriger, comme le demande le prompt système. Les docstrings des outils servent directement

de description pour le LLM : elles précisent le format attendu (par exemple « 0.8 for 80% »).

Évaluation du chatbot. `chatbot_tests.json` contient 20 requêtes en français : 15 demandent un seul outil (3 par outil) et 5 en enchaînent deux, avec les outils et arguments attendus. `eval_chatbot.py` mesure si l'ensemble des outils appelés est le bon, puis si les arguments le sont aussi. Les noms de races sont comparés avec la même recherche floue que le serveur, et les seuils avec tolérance au format 0,9 ou 90.

qwen2.5 :7b, 20 requêtes	**Score**
Bons outils	90 % (18/20)
Bons outils + bons arguments	85 % (17/20)
Requêtes à 1 outil	93 % (14/15)
Requêtes enchaînées	60 % (3/5)

Tab. 6. Évaluation du chatbot (`outputs/chatbot_eval_qwen2.5_7b.txt`).

Les appels simples sont presque toujours corrects, y compris la conversion des pourcentages (« plus de 90 % » → `min_confidence: 0.9`) et la normalisation des noms (« chats persans » → Persian). Les trois échecs sont instructifs :

- **« Combien de Sphynx le modèle a-t-il trouvés ? »** : le LLM appelle `filter_images` avec un seuil de 0,8 qu'il a inventé, au lieu de `count_by`. Le nombre annoncé (181) correspond donc aux images prédites avec plus de 80 % de confiance, et non au total.
- **Ragdoll puis images à plus de 90 %** : après `most_confused`, le LLM n'appelle pas `filter_images`. Il présente les exemples d'erreurs renvoyés par le premier outil comme des « images prédites à plus de 90 % » : la réponse a l'air plausible, mais elle est fausse.
- **Egyptian Mau puis comptage** : les deux bons outils sont appelés, mais le LLM compte « Egyptian Mau » au lieu de la race trouvée par le premier outil (Bengal). Il ne réutilise pas le résultat intermédiaire.

Les requêtes enchaînées restent le point faible d'un modèle de 7 milliards de paramètres (60 %). En revanche, quand la seconde étape porte sur la même entité que la première (par exemple `classify` puis `find_similar` sur la même image), l'enchaînement fonctionne.

4. Développement : difficultés rencontrées et solutions

Temps d'entraînement. Le protocole demande 6 entraînements complets (2 variantes × 3 seeds), dont 3 from scratch de 30 époques chacun, sur 3 100 images de 224×224. L'ensemble représente un coût de calcul important et impose un GPU pour rester dans des délais raisonnables. Nous l'avons géré de plusieurs façons :

- un script `run_all.bat` enchaîne les 6 runs sans intervention ;
- `get_device()` choisit automatiquement CUDA, MPS ou CPU, pour que le même code tourne sur toutes nos machines ;
- on ne garde que le meilleur checkpoint de validation, et `evaluate.py` réévalue les checkpoints sauvegardés : produire les résultats, les figures et les fichiers du serveur MCP ne demande pas de réentraîner ;
- le nombre d'époques est réduit pour le modèle pré-entraîné (10), qui converge vite grâce au transfert.

Partage des checkpoints. Les 6 checkpoints pèsent 43 Mo chacun (environ 260 Mo au total). Ils ont d'abord été exclus du dépôt car trop volumineux. Nous les avons finalement ajoutés au dépôt Git pour que chaque membre et le correcteur puissent utiliser le chatbot et réévaluer les modèles sans tout réentraîner.

Portabilité Windows → Mac/Linux. L'index `index.csv` avait été généré sous Windows : les chemins des images contenaient des barres obliques inverses (`data\oxford-iiit-pet\images\...`). Sous Mac et Linux, `pathlib` ne les interprète pas comme des séparateurs. Le nom de fichier extrait était donc faux, et la recherche d'une image par son nom (« Abyssinian_100 ») échouait.

La correction normalise les chemins au chargement dans le serveur (`replace("\\", "/")`) et dans le script d'évaluation du chatbot. De plus, `evaluate.py` écrit désormais les chemins au format POSIX (`as_posix()`), quel que soit le système.

Images indisponibles. Le jeu de données (dossier `data/`) n'est pas versionné. Lors des tests finaux, le serveur de l'université d'Oxford qui l'héberge ne répondait plus, et l'outil `classify`, qui relit l'image pour la passer dans le modèle, plantait. Nous avons ajouté un repli : si le fichier image est absent, l'outil renvoie la prédiction déjà calculée par le même modèle dans `index.csv` (race et confiance), avec une note indiquant que le top-3 n'est pas disponible. Le chatbot reste ainsi utilisable sans le jeu de données.

5. Conclusion

Nous avons réalisé un assistant complet qui répond en langage naturel à des questions sur une collection de 7 349 images. Le ResNet-18 pré-entraîné atteint 90,0 ± 0,2 % de top-1 et 89,3 % de precision@5 sur le test, loin devant le même réseau entraîné from scratch (40,9 %) et les baselines. Le chatbot basé sur qwen2.5 :7b choisit le bon outil avec les bons arguments dans 85 % des cas.

5.1 Compétences acquises

- **Transfer learning** : fine-tuning d'un réseau pré-entraîné (taux d'apprentissage différenciés, augmentation de données, sélection sur validation) et mesure de son apport par une ablation.
- **Évaluation rigoureuse** : plusieurs seeds avec moyenne ± écart-type, baselines, métriques de classification et de recherche, analyse par classe et analyse d'erreurs.
- **LLM et outils** : protocole MCP, conception d'outils et de descriptions lisibles par un LLM, prompt système, boucle d'appels d'outils, et évaluation automatique d'un agent.
- **Travail en équipe et génie logiciel** : collaboration avec Git, portabilité entre systèmes, robustesse face aux entrées imprécises et aux ressources manquantes.

5.2 Pistes d'amélioration

- **Enchaînements du LLM** : ajouter au prompt des exemples d'enchaînements (*few-shot*), comparer avec des modèles plus grands (llama3.1 :8b sur le serveur du CRIL, modèles de 14 milliards de paramètres ou plus) et renommer le champ `example_errors` de `most_confused` pour éviter qu'il soit pris pour une liste d'images bien classées.
- **Modèle de vision** : backbone plus puissant (ResNet-50, ViT), augmentations plus fortes, ou entraînement ciblé sur les paires confondues (bull-terriers, Birman/Ragdoll). L'option `--no-augment` existe déjà pour une ablation de l'augmentation que nous n'avons pas eu le temps de mener.
- **Calibration** : le modèle peut se tromper avec 100 % de confiance. Un *temperature scaling* rendrait les seuils de confiance de `filter_images` plus fiables.
- **Serveur MCP** : stocker le top-3 dans `index.csv` pour que `classify` soit complet même sans les images, et ajouter des outils (recherche par texte, statistiques par split).

– **Évaluation du chatbot** : élargir le jeu de requêtes (formulations ambiguës, erreurs volontaires) et vérifier aussi l'exactitude des réponses finales, pas seulement les appels d'outils.